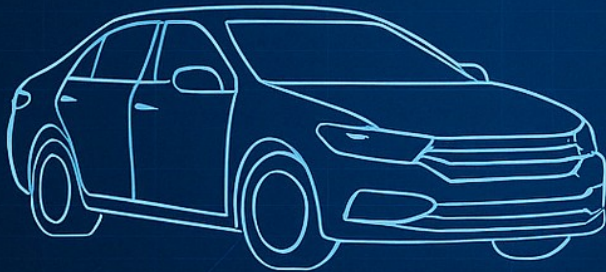


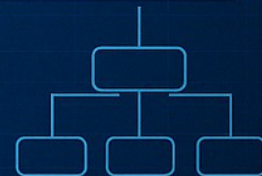
PROYECTO DE BASE DE DATOS

Sistema de Alquiler de Vehículos

CarRentalX



```
SELECT * FROM Vehiculos  
WHERE disponible = i;
```



Informe Final

Tabla de contenidos

Introducción.....	3
El Proyecto de Base de Datos de CarRentalX.....	3
Análisis del Enunciado.....	3
Modelo Conceptual.....	4
Principales entidades en el modelo conceptual.....	4
Relaciones en el modelo conceptual.....	4
Modelo Relacional.....	6
Descripción de las tablas del modelo relacional.....	6
Relaciones entre tablas del modelo relacional.....	6
Integridad y Normalización en el diagrama de MySQL Workbench.....	7
Implementación en MySQL.....	9
Importación del esquema de la base de datos.....	9
Inserción de Datos.....	11
Consultas propuestas.....	12
Consultas básicas.....	12
Consultas de modificación y mantenimiento.....	12
Consultas con filtros.....	12
Consultas con JOIN.....	12
Consultas con funciones agregadas.....	12
Consultas con ordenado, fechas y RANK.....	13
Ampliación y Mejoras de la Base de Datos.....	14
Vistas y triggers.....	15
Vistas.....	15
Triggers.....	15
Trigger liberar_vehiculo_al_completar.....	15
Trigger actualizar_mantenimiento.....	16
Pruebas y validación.....	17
Conclusiones.....	18
Anexos.....	19



Introducción

El Proyecto de Base de Datos de CarRentalX

Este informe presenta el desarrollo completo del proyecto de base de datos CarRentalX, realizado como parte del módulo de **Bases de Datos en 1º de DAM**. El objetivo ha sido diseñar, construir e implementar una base de datos funcional, siguiendo un enunciado común y documentando cada etapa del proceso de forma clara y ordenada.

A lo largo del proyecto se han seguido todas las fases planteadas: desde el análisis inicial y el diseño del modelo conceptual, pasando por la creación del modelo relacional y su implementación en MySQL. Posteriormente, se llevó a cabo la inserción de datos, la ejecución de consultas de prueba, y la creación de vistas y triggers para mejorar el acceso a la información y automatizar procesos.

También se amplió el sistema con nuevas funcionalidades, como el historial de salarios. Todo este trabajo ha quedado documentado en el informe y acompañado por los scripts y recursos organizados en el repositorio de GitHub del proyecto.

El resultado es un sistema que cubre las principales necesidades de una empresa de alquiler de vehículos, y que además ha sido diseñado para poder crecer y adaptarse fácilmente a nuevas funcionalidades en el futuro.

Análisis del Enunciado

El desarrollo del proyecto se ha basado en las instrucciones proporcionadas en la carpeta **Proyectos de la Base de Datos** del repositorio del curso. Desde el inicio, se respetó la estructura recomendada: se creó el archivo informe.md para documentar el trabajo, se organizaron los scripts en su carpeta correspondiente (creacion.sql, insercion.sql, consultas.sql, vistas-triggers.sql, ampliacion.sql), y se incluyeron recursos adicionales como capturas y ejemplos de prueba.

Cada una de las etapas del enunciado fue desarrollada en orden lógico, siguiendo las prácticas aprendidas en clase. Además, se utilizó Git y GitHub como herramienta de control de versiones, trabajando en ramas separadas para mantener un flujo de trabajo ordenado y fácil de seguir.

En general, en el proyecto se han cumplido todos los requisitos planteados en el enunciado.

Modelo Conceptual

El modelo escogido está orientado a la gestión de la empresa CarRentalX de alquiler de vehículos y contempla los distintos elementos y procesos que intervienen en su operativa.

Como primer paso en la elaboración de la base de datos se debe realizar el modelo conceptual. Este se elaboró utilizando el enfoque Entidad-Relación (E-R), lo permite una representación clara y detallada de la estructura lógica de la base de datos.

Principales entidades en el modelo conceptual

Las principales entidades que se han contemplado en la base de datos son por un lado, tablas para los clientes y las reservas, y por el otro, tablas para la gestión interna como los vehículos, las sucursales, los empleados y las reservas:

- **VEHÍCULOS:** Cada vehículo está identificado por una matrícula única y posee atributos como modelo, año, tipo, combustible, transmisión, precio diario, y kilometraje, entre otros. Se encuentra relacionado con varias entidades, lo que refleja su centralidad en la operativa del sistema.
- **SUCURSAL:** Representa las distintas ubicaciones físicas donde opera la empresa. Aporta información administrativa (dirección, teléfono, horarios, ingresos mensuales) y se vincula tanto con vehículos como con empleados.
- **EMPLEADO:** Incluye datos personales, de contacto y laborales. Cada empleado puede gestionar reservas y estar asignado a una sucursal específica.
- **CLIENTES:** Recoge los datos de los usuarios que realizan reservas. Se incluyen campos relevantes como nombre, contacto, número de licencia y fechas de vigencia.
- **RESERVAS:** Esta entidad representa las operaciones comerciales de alquiler. Contempla aspectos clave como fechas, estado, método de pago, observaciones y vínculo con vehículo, cliente y empleado responsable.
- **MANTENIMIENTO:** Controla los servicios realizados a los vehículos, incluyendo el tipo, costo, kilometraje y fechas. Se vincula tanto al vehículo como al empleado que lo gestiona, aportando trazabilidad al historial técnico.

Relaciones en el modelo conceptual

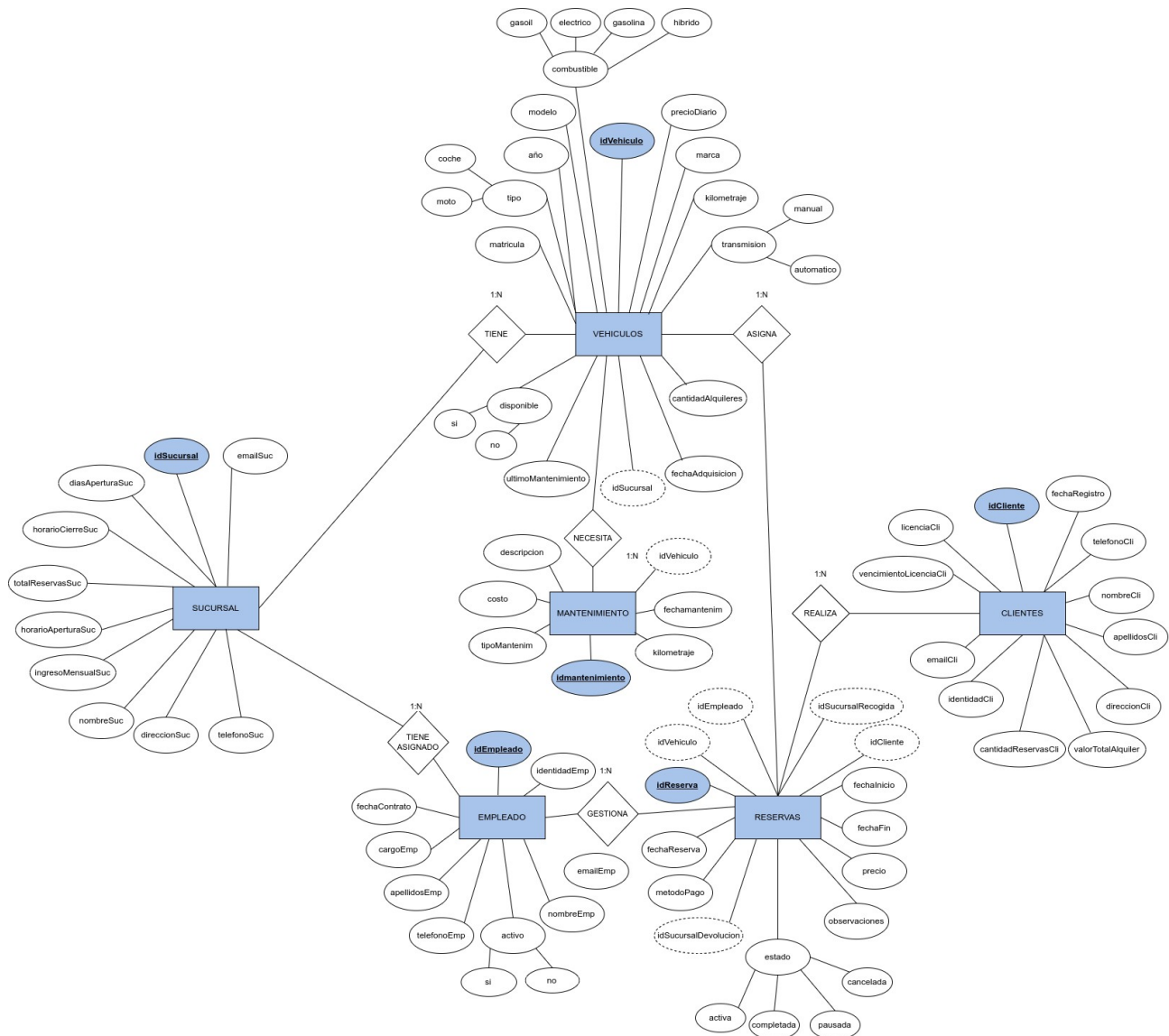
Para que estas tablas funcionen de manera orgánica, necesitan relacionarse entre ellas, estas son las principales relaciones que se contemplan en el modelo conceptual:

- **TIENE** (Sucursal–Vehículo): Indica la ubicación actual de cada vehículo, lo que es fundamental para la disponibilidad.
- **REALIZA** (Cliente–Reserva): Registra qué cliente ha efectuado qué reserva, con una relación 1:N.
- **GESTIONA** (Empleado–Reserva): Asocia al empleado que ha tramitado una reserva concreta.

- **NECESITA** (Vehículo–Mantenimiento): Refleja la relación directa entre el estado técnico del vehículo y su mantenimiento.
- **TIENE ASIGNADO** (Sucursal–Empleado): Define qué empleados están vinculados a cada sucursal.

Para que la relación funcione de manera adecuada, se crean también conceptualmente las llaves primarias y llaves foráneas (idVehiculo, idCliente, etc.) que son necesarias para que funcionen las relaciones.

Tras la creación conceptual, pasamos a la creación del modelo relacional, donde plasmamos en MySQL Workbench el modelo conceptual de forma más específica y con función operativa.



Modelo Relacional

El modelo relacional se basa en el modelo conceptual, transformando entidades y relaciones en tablas normalizadas, y respetando las reglas de integridad referencial mediante claves primarias y foráneas. Esta fase representa la transición lógica entre el análisis y la implementación física en un sistema gestor de bases de datos, en este caso MySQL Workbench, donde después pasará a ser importado de forma efectiva como una base de datos real a través de un script generado automáticamente por el programa.

A la hora de elaborar el esquema, el programa corrige de forma semiautomática cualquier error de concepto, por lo que el modelo conceptual de CarRentalX no sufre errores que pudieran heredarse del modelo conceptual.

Descripción de las tablas del modelo relacional

- **vehículos:** Contiene todos los datos asociados a cada vehículo, como tipo, marca, modelo, año, combustible, transmisión, kilometraje y estado de disponibilidad...
- **clientes:** Registra la información personal y de contacto de los clientes. Se incluyen campos como licencia de conducir, cantidad de reservas realizadas y la fecha de registro. Se utiliza idCliente como clave primaria.
- **empleados:** Almacena datos personales y laborales de los empleados, tales como nombre, identificación, cargo, correo y estado de actividad. La relación con otras tablas permite controlar su participación en reservas y asignación a sucursales.
- **sucursales:** Tabla que centraliza la gestión de las distintas ubicaciones físicas. Se registra nombre, dirección, horarios, contacto y estadísticas como ingresos mensuales y total de reservas.
- **reservas:** Es una de las tablas más importantes, ya que representa la operación principal del sistema. Contiene información clave como fechas de inicio y fin, vehículo alquilado, cliente, empleado que la gestionó, sucursales de recogida y devolución, estado de la reserva y método de pago. Sus claves foráneas son de las más importantes porque reflejan las relaciones complejas que tiene con otras entidades.
- **mantenimientos:** Controla los servicios realizados a cada vehículo. Se registra el tipo, fecha, costo, descripción y kilometraje asociado al mantenimiento.

Relaciones entre tablas del modelo relacional

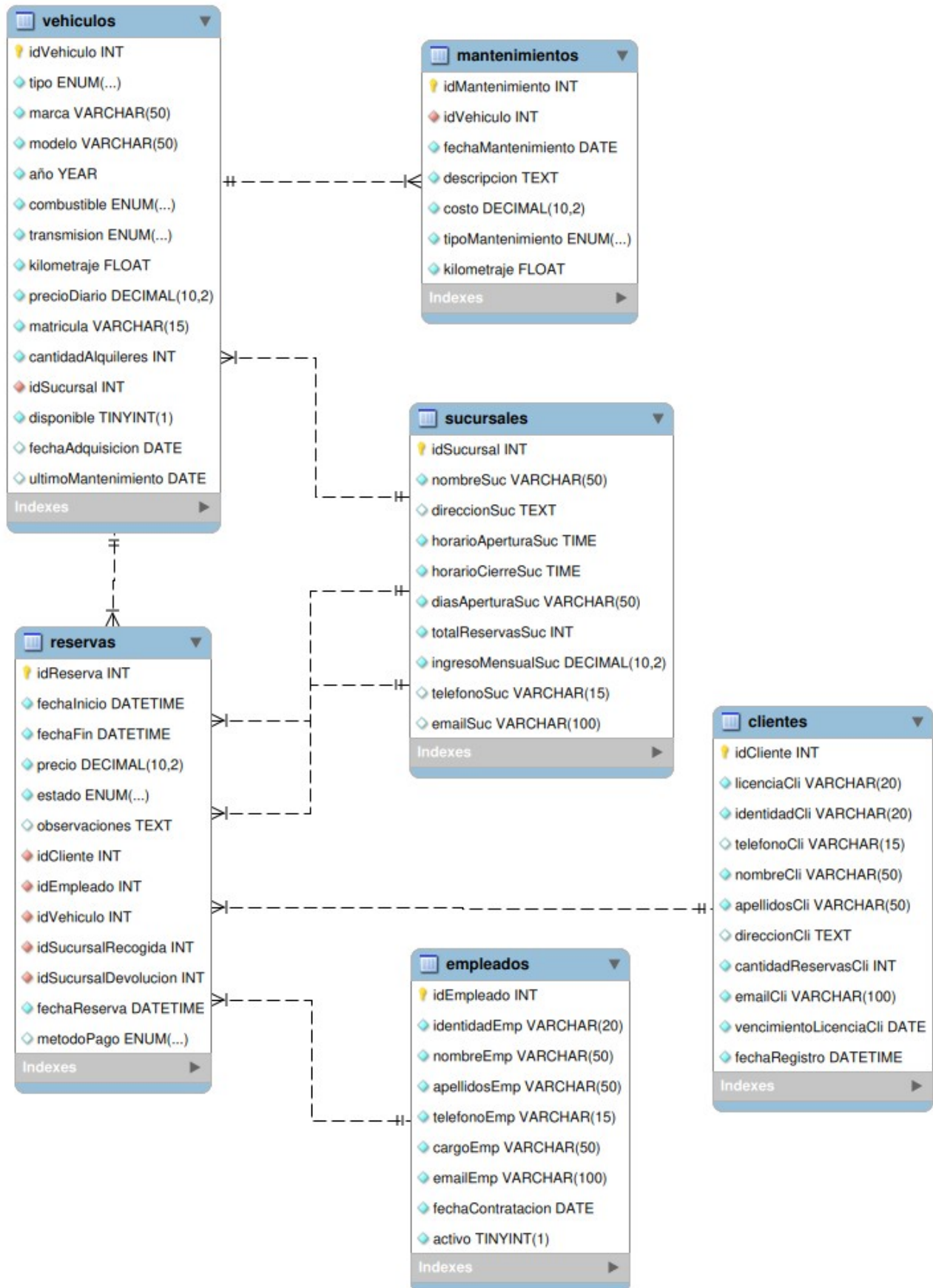
A continuación paso a definir las principales relaciones que se tienen en cuenta a la hora de diseñar la base de datos de CarRentalX:

- Cada **vehículo** está asociado a una única **sucursal** a través del campo idSucursal, lo que permite conocer en todo momento en qué sede se encuentra disponible.
- Los **vehículos** pueden tener múltiples **mantenimientos** registrados, vinculados mediante el campo idVehiculo en la tabla mantenimientos. Permite llevar un historial técnico detallado de cada transporte.

- Los **clientes** pueden realizar múltiples **reservas**, y esta relación se implementa con el campo `idCliente` en la tabla `reservas`. De este modo, se puede hacer un seguimiento y generar informes o historial de alquileres.
- Cada **reserva** es gestionada por un **empleado** específico, lo que se refleja en la relación entre `reservas` y `empleados` mediante `idEmpleado`.
- Un **vehículo** puede estar implicado en muchas **reservas** diferentes a lo largo del tiempo. Esta relación está expresada mediante `idVehiculo` en la tabla `reservas`, y sirve para conocer cuántas veces ha sido alquilado un mismo coche o moto.
- Las **reservas** también están vinculadas con dos **sucursales distintas**: una para la recogida (`idSucursalRecogida`) y otra para la devolución (`idSucursalDevolucion`). Esto permite gestionar escenarios en los que el cliente devuelve el vehículo en un lugar distinto al de origen.

Integridad y Normalización en el diagrama de MySQL Workbench

El diseño de las tablas tiene en cuenta la clave primaria y se usan claves foráneas para mantener las relaciones entre datos sin errores. Los tipos de datos también se han adecuado al tipo de dato que se registra, por ejemplo, se usan `TEXT` para descripciones largas y `DECIMAL` o `FLOAT` para precios o kilometrajes. Además, el uso de `ENUM` en campos como el tipo de vehículo, estado de reserva o método de pago ayuda bastante a evitar errores, ya que limita las opciones a valores válidos. Lo mejor es que todo está organizado de forma que no se repite información innecesaria, lo que facilita mucho hacer cambios o añadir nuevas funcionalidades más adelante.

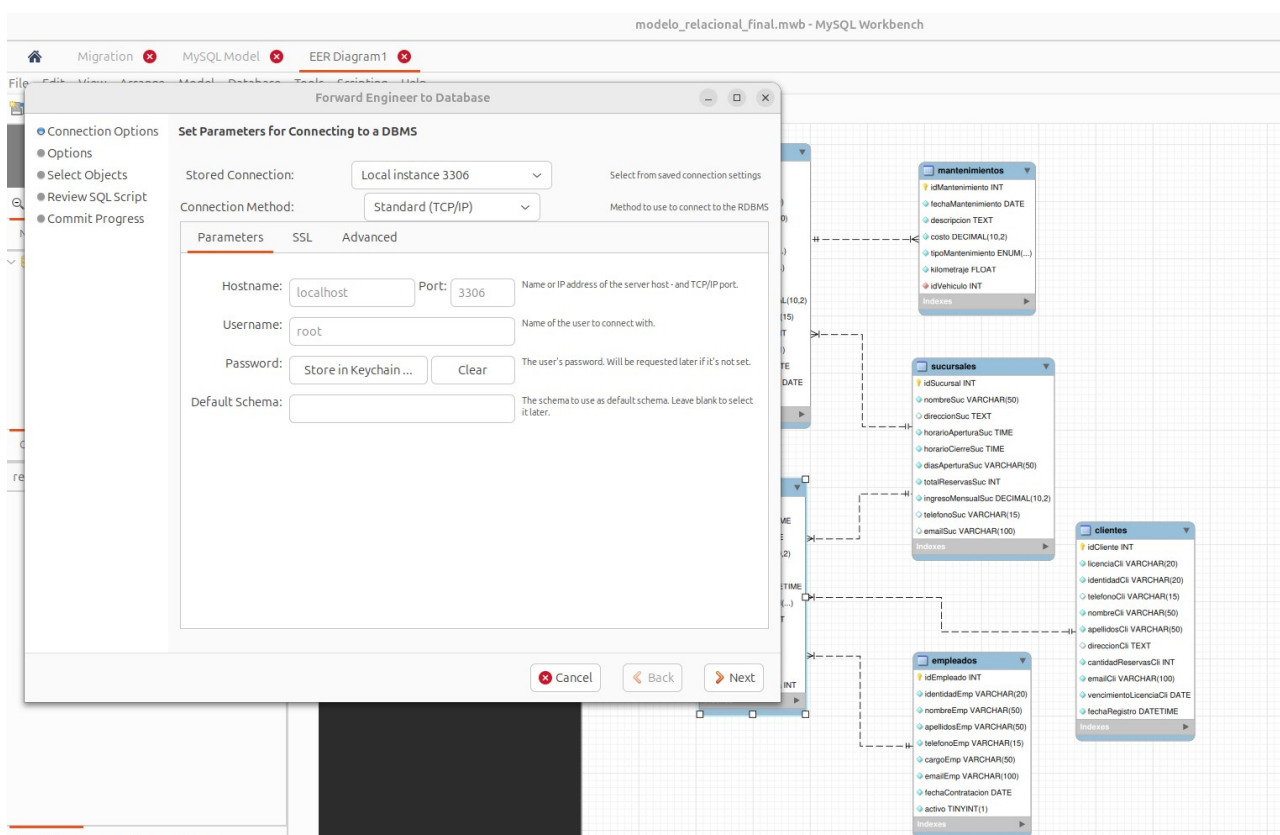


Implementación en MySQL

Importación del esquema de la base de datos

Una vez definido el modelo relacional para la empresa de alquiler CarRentalX, el siguiente paso es su implementación en MySQL. En este caso, se ha utilizado el software **MySQL Workbench**, que es una herramienta visual que facilita la creación, edición y despliegue de esquemas.

A partir del diagrama relacional, se construyeron las tablas con sus respectivos atributos, tipos de datos y restricciones. El programa permite generar automáticamente el script SQL necesario para crear la base de datos y sus relaciones, trasladando el diseño conceptual a un entorno real donde se podrán ejecutar las consultas, insertar datos, y en definitiva hacer efectiva la base de datos.



Se procedió a su implementación en MySQL utilizando un script SQL generado y afinado desde MySQL Workbench con la herramienta **Forward Engineer to Database**. El archivo incluye la creación de todas las tablas, cada una con sus respectivos campos, tipos de datos y restricciones de integridad. Se hace uso de claves primarias auto incrementales, claves foráneas para mantener las relaciones entre entidades, y restricciones UNIQUE para evitar duplicados en campos clave como licencias, correos electrónicos o identificaciones.

Además, se utiliza el motor InnoDB y la codificación utf8mb4 que asegura la compatibilidad con caracteres especiales y acentos latinos.

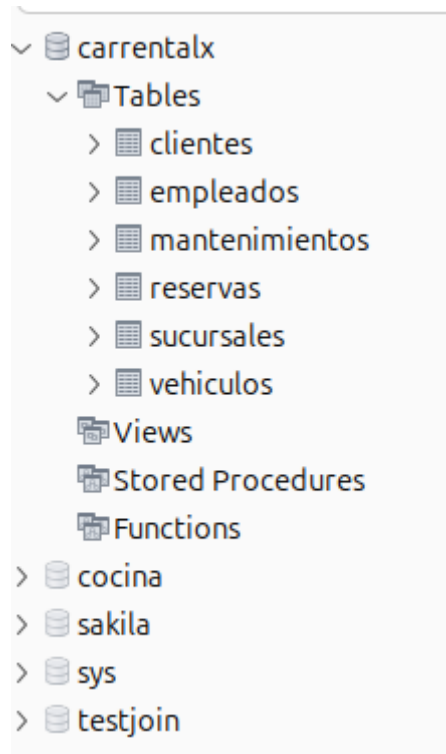
Un ejemplo de tabla dentro del script es la de clientes, que define los datos personales de los usuarios del sistema. El fragmento siguiente muestra cómo se crea esta tabla, el **script completo se puede consultar en el proyecto de Github**, en la carpeta scripts:

```

1  -- MySQL Workbench Forward Engineering
2
3  ● SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
4  ● SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
5  ● SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLES,NO_ZERO_IN_DATE,NO_ZERO_DATE,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION';
6
7  -----
8  -- Schema carrentalx
9  -----
10
11 -----
12 -- Schema carrentalx
13 -----
14 ● CREATE SCHEMA IF NOT EXISTS `carrentalx` DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci ;
15 ● USE `carrentalx` ;
16
17 -----
18 -- Table `carrentalx`.`clientes`
19 -----
20 ● ○ CREATE TABLE IF NOT EXISTS `carrentalx`.`clientes` (
21     `idCliente` INT NOT NULL AUTO_INCREMENT,
22     `licenciaCli` VARCHAR(20) NOT NULL,
23     `identidadCli` VARCHAR(20) NOT NULL,
24     `telefonoCli` VARCHAR(15) NULL DEFAULT NULL,
25     `nombreCli` VARCHAR(50) NOT NULL,
26     `apellidosCli` VARCHAR(50) NOT NULL,
27     `direccionCli` TEXT NULL DEFAULT NULL,
28     `cantidadReservasCli` INT NOT NULL DEFAULT '0',
29     `emailCli` VARCHAR(100) NOT NULL,
30     `vencimientoLicenciaCli` DATE NOT NULL,
31     `fechaRegistro` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
32     PRIMARY KEY (`idCliente`),
33     UNIQUE INDEX `licenciaCli_UNIQUE` (`licenciaCli` ASC) VISIBLE,
34     UNIQUE INDEX `identidadCli_UNIQUE` (`identidadCli` ASC) VISIBLE,
35     UNIQUE INDEX `emailCli_UNIQUE` (`emailCli` ASC) VISIBLE)
36     ENGINE = InnoDB
37     DEFAULT CHARACTER SET = utf8mb4
38     COLLATE = utf8mb4_unicode_ci;

```

Una vez ejecutado el script SQL, se crea de facto el esquema de la base de datos dentro de MySQL:



Inserción de Datos

Una vez creada la estructura de la base de datos, el siguiente paso es poblarla con datos iniciales que permitan realizar pruebas y validar su funcionamiento. Para ello, se preparó un script SQL que inserta registros en todas las tablas principales del sistema CarRentalX, incluyendo sucursales, empleados, clientes, vehículos, mantenimientos y reservas.

Estos datos se incluyen para simular datos reales y están pensados para verificar que las relaciones entre tablas se mantienen correctamente y que el sistema es capaz de operar con datos coherentes y en la formatación que se ha escogido.

Para evitar mensajes de error, las sentencias INSERT INTO se organizaron con lógica para evitar errores: primero se llenaron las tablas más “independientes”, como las de sucursales y clientes, y después se fueron completando las que dependen de esas, como vehículos, reservas o mantenimientos. De esta forma, todo encaja bien desde el principio y no hay problemas con las claves foráneas.

Esta muestra es parte del script que se usó, que **puede consultarse de forma completa en la carpeta de scripts del proyecto en Github**:

```
1 • USE carrentalx;
2 -- Insertar las sucursales
3 • INSERT INTO `carrentalx`.`sucursales`
4   (`nombreSuc`, `direccionSuc`, `horarioAperturaSuc`, `horarioCierreSuc`, `diasAperturaSuc`, `telefonoSuc`, `emailSuc`) VALUES
5   ('Sucursal Centro', 'Av. Principal 123, Centro', '08:00:00', '20:00:00', 'Lunes-Sábado', '922-280234', 'centro@carrentalx.com'),
6   ('Sucursal Aeropuerto', 'Terminal 2, Aeropuerto Internacional', '06:00:00', '23:00:00', 'Lunes-Domingo', '699-885544', 'aeropuerto@carrentalx.com'),
7   ('Sucursal Norte', 'Plaza Comercial Norte, Local 45', '09:00:00', '21:00:00', 'Lunes-Domingo', '935-456789', 'norte@carrentalx.com'),
8   ('Sucursal Playa', 'Blvd. Costero 789, Zona Turística', '07:00:00', '22:00:00', 'Lunes-Domingo', '691-386756', 'playa@carrentalx.com');
9
10 -- Insertar los empleados de CarRentx
11 • INSERT INTO `carrentalx`.`empleados`
12   (`identidadEmp`, `nombreEmp`, `apellidosEmp`, `telefonoEmp`, `cargoEmp`, `emailEmp`, `fechaContratacion`) VALUES
13   ('78706463W', 'Carlos', 'Ramírez López', '922-280234', 'Gerente', 'carlos.ramirez@carrentalx.com', '2020-06-15'),
14   ('78706465E', 'Ana', 'Martínez Sánchez', '935-456789', 'Recepcionista', 'ana.martinez@carrentalx.com', '2021-03-10'),
15   ('78706478J', 'Roberto', 'González Pérez', '699-885544', 'Mecánico', 'roberto.gonzalez@carrentalx.com', '2022-01-05'),
16   ('78706120T', 'Sofía', 'Hernández Díaz', '691-386756', 'Ejecutiva de Ventas', 'sofia.hernandez@carrentalx.com', '2021-09-20');
17
18 -- Insertar 4 registros de clientes
19 • INSERT INTO `carrentalx`.`clientes`
20   (`licenciaCli`, `identidadCli`, `telefonoCli`, `nombreCli`, `apellidosCli`, `direccionCli`, `emailCli`, `vencimientoLicenciaCli`) VALUES
21   ('78706463W', '78706463W', '922-280234', 'Miguel', 'Torres García', 'Calle Roble 45, Colonia Vista', 'miguel.torres@email.com', '2026-05-20'),
22   ('78706465E', '78706465E', '699-885544', 'Laura', 'Méndez Ruiz', 'Av. Pinos 78, Residencial Las Palmas', 'laura.mendez@email.com', '2025-11-15'),
23   ('78706478J', '78706478J', '935-456789', 'Javier', 'Ortega Vega', 'Blvd. Central 120, Edificio Azul', 'javier.ortega@email.com', '2024-08-30'),
24   ('78706120T', '78706120T', '691-386756', 'Patricia', 'Luna Castro', 'Calle Lirios 33, Fracc. Jardines', 'patricia.luna@email.com', '2025-03-25');
```

Consultas propuestas

Este apartado recoge una serie de consultas SQL diseñadas para validar el funcionamiento y la consistencia de la base de datos **CarRentalX**.

Estas consultas no solo sirven para probar que los datos fueron insertados correctamente, sino también como demostración práctica del manejo de SQL sobre el esquema implementado. Se trata de un conjunto variado de sentencias que permiten recuperar información relevante, analizar el comportamiento del sistema y detectar posibles inconsistencias.

Las consultas incluidas fueron pensadas como pruebas funcionales y ejercicios de práctica, abarcando desde operaciones básicas como listar todos los vehículos o clientes, hasta otras más específicas que permiten unir varias tablas o aplicar filtros más complejos.

Consultas básicas

Las consultas más simples del script están orientadas a mostrar todo el contenido de algunas tablas clave, utilizando sentencias `SELECT *` son ideales para tener una vista general y detectar errores obvios o valores nulos.

Consultas de modificación y mantenimiento

Como primer paso, en el script se incluyen sentencias destinadas a modificar tanto los datos como la estructura de algunas tablas

Las sentencias `UPDATE` se utilizan para modificar registros existentes. Son útiles para validar que los datos se pueden actualizar correctamente y que las restricciones definidas (como claves foráneas o restricciones `UNIQUE`) se mantienen consistentes.

Por otro lado, las sentencias `ALTER TABLE` permiten modificar la estructura de una tabla ya existente. En el script se utilizan para agregar nuevas columnas, ajustar tipos de datos o incluso añadir restricciones que no se habían definido inicialmente.

Consultas con filtros

El script incluye consultas más específicas, utilizando cláusulas `WHERE` para filtrar resultados. Permiten obtener subconjuntos de datos útiles, como vehículos disponibles, reservas activas o empleados que trabajan en una determinada sucursal.

Consultas con JOIN

Uno de los puntos del script es la inclusión de consultas que combinan múltiples tablas usando `INNER JOIN`, `LEFT JOIN`, etc. Estas permiten cruzar información, como ver qué cliente hizo qué reserva, o qué empleados están asignados a qué vehículos para tareas de mantenimiento. Estas consultas son fundamentales para obtener información compuesta entre tablas.

Consultas con funciones agregadas

El script también incluye consultas que hacen uso de funciones como `COUNT`, `AVG`, `SUM` o `GROUP BY`, lo que permite generar estadísticas simples, como la cantidad de reservas por cliente, el total de ingresos por sucursal o el promedio de mantenimientos por vehículo.

Consultas con ordenado, fechas y RANK

El script también incorpora consultas que hacen uso de ordenamientos mediante ORDER BY, y filtros y operaciones con fechas utilizando funciones como NOW(), BETWEEN o DATEDIFF. Son especialmente útiles para obtener resultados ordenados por antigüedad, identificar licencias que van a vencer, mostrar reservas activas dentro de un rango de fechas o listar los vehículos más recientes.

También se aplican funciones de ranking como RANK() para generar clasificaciones dentro de subconjuntos de datos, por ejemplo, para saber qué clientes han realizado más reservas o qué vehículos han generado más ingresos.

A continuación se muestra parte del script que se realizó, este **se encuentra comentado y de forma integral dentro de la carpeta de scripts del proyecto en Github:**

```
1  -- Usar la base de datos CarrentalX
2
3  • USE carrentalx;
4
5  -- Voy a alterar varios campos de la base de datos o incluir nueva información:
6
7  -- Voy a alterar el campo disponibilidad de TYNINT a ENUM de la tabla vehiculos para dejarla más clara y adecuada.
8  -- Como ya esta creada y rellena, voy a crear una tabla temporal, cambiar algunos valores y pasarlos a la nueva columna.
9
10 • ALTER TABLE vehiculos ADD COLUMN disponible_tmp ENUM('Disponible', 'No disponible') DEFAULT 'Disponible';
11 • SELECT * FROM vehiculos;
12
13 -- Cambio dos campos a 0 para que haya variedad
14
15 • UPDATE vehiculos SET disponible=0 WHERE tipo IN ('Moto','Furgoneta');
16
17 -- Paso la información de cero como No disponible y 1 como Disponible a la nueva columna con una sentencia CASE:
18
19 • UPDATE vehiculos
20   SET disponible_tmp = CASE
21     WHEN disponible = 1 THEN 'Disponible'
22     WHEN disponible = 0 THEN 'No disponible'
23   END;
24
25 -- Borro la columna antigua y renombro la nueva
26
27 • ALTER TABLE vehiculos DROP COLUMN disponible;
28 • ALTER TABLE vehiculos CHANGE disponible_tmp disponible ENUM('Disponible', 'No disponible') DEFAULT 'Disponible';
```

Ampliación y Mejoras de la Base de Datos

Como parte de la mejora de la base de datos **CarRentalX**, se ha incorporado una ampliación que permite gestionar el salario y el historial de salarios de los empleados. La mejora tiene como objetivo crear una trazabilidad en los cambios de contrato y los salarios de los empleados

La ampliación comienza en el script mediante la creación de una nueva tabla llamada **salarios**, que almacena el salario actual de cada empleado, junto con la fecha de inicio de ese salario. La tabla está relacionada directamente con la entidad empleados, utilizando idEmpleado como clave foránea. Después de crearse, se insertaron registros representando los salarios iniciales de los empleados, con fechas específicas de inicio.

Para mejorar aún más, se ha implementado un **trigger** que actúa automáticamente cuando se actualiza el salario de un empleado. Este trigger guarda el salario anterior en una tabla adicional de **histórico de salarios**, junto con la fecha del cambio. Así, se puede consultar en cualquier momento cuánto ganaba un empleado anteriormente y cuándo se modificó su sueldo, sin perder información histórica.

Como comentado, para que el trigger funcione correctamente, se hizo necesario crear previamente otra tabla adicional llamada **salarios_anterior**, que actúa como un historial de cambios, registrando el idEmpleado, el salario anterior y la fecha en que se produjo la modificación.

A continuación se muestra parte del script que fue realizado para crear las dos tablas y el trigger de mejora de la base de datos, para ver **el script completo se debe acudir a la carpeta scripts del proyecto en Github**.

```
1  # Mejoras a la base de datos creando la tabla salarios,
2  # que se complementa con una tabla de registro de salarios anteriores y un trigger para guardar la fecha y el salario anterior
3
4  • USE carrentalx;
5
6  # Se va a crear una tabla de salarios actuales de los empleados
7  # que va a permitir ver cuando se inicia el contrato y el valor del salario.
8
9  • CREATE TABLE `salarios` (
10     `idSalario` INT NOT NULL AUTO_INCREMENT,
11     `idEmpleado` INT NOT NULL,
12     `salario` DECIMAL(10,2) NOT NULL,
13     `fechaInicio` DATE NOT NULL,
14     PRIMARY KEY (`idSalario`),
15     FOREIGN KEY (`idEmpleado`) REFERENCES `empleados`(`idEmpleado`) ON DELETE CASCADE
16 );
17
18 -- Y le metemos inserciones sobre el salario de los 4 empleados
19
20 • INSERT INTO `salarios` (`idEmpleado`, `salario`, `fechaInicio`)
21 VALUES
22 (1, 1800.00, '2024-01-01'),
23 (2, 1950.50, '2023-11-15'),
24 (3, 2000.00, '2025-01-01'),
25 (4, 1500.90, '2022-09-09');
26
27 • SELECT * FROM salarios;
```

Vistas y triggers

Vistas

Las **vistas** simplifican consultas complejas al encapsularlas en una tabla temporal. De esta forma, se facilita la visualización de datos relevantes para distintos perfiles de usuario sin tener que escribir consultas SQL largas o complicadas.

El script define varias vistas orientadas tanto a la gestión operativa como al análisis de datos.

- **vista_clientes_vehiculos_alquilados**: Esta vista combina las tablas de clientes, reservas y vehículos para mostrar qué cliente alquiló qué vehículo y en qué fechas. Es muy útil para generar informes o ver el historial sin necesidad de múltiples joins.
- **vista_vehiculos_mas_alquilados**: Agrupa las reservas por vehículo y muestra cuántas veces ha sido alquilado cada uno.
- **vista_ingresos_por_vehiculo**: Calcula el total de ingresos generados por cada vehículo, sumando el precio de todas sus reservas. Es útil para saber cuanto genera cada vehículo de la flota.
- El script incluye otras vistas muy útiles tales como **vista_empleados_activos_por_sucursal**, **vista_reservas_completadas**, y **vista_vehiculos_disponibles** que pueden ser consultadas al completo en la carpeta de scripts del proyecto de Github.

Además, se hizo necesario que el script contemplase la necesidad de mejorar ciertas relaciones antes de crear más vistas, como añadir el campo **idSucursal** en la tabla empleados para reflejar correctamente la asignación del personal a sucursales.

Triggers

Los **triggers**, como ya se vio en el apartado de Ampliación y Mejoras, permiten ejecutar acciones automáticas cuando ocurren ciertos eventos (como una inserción o actualización), lo que aporta dinamismo y control a la base de datos sin necesidad de intervención manual.

Dentro del script se han implementado dos triggers útiles:

Trigger liberar_vehiculo_al_completar

Este trigger se ejecuta **después de una actualización** en la tabla reservas, y su objetivo es actualizar automáticamente el estado de disponibilidad de un vehículo cuando una reserva se marca como *completada*: Si el estado anterior era distinto a "Completada" y ahora se actualiza a ese valor, entonces se modifica el campo disponible del vehículo correspondiente, marcándolo como "Disponible" nuevamente.

El trigger se probó directamente actualizando la reserva de una moto Yamaha, cambiando su estado de *Activa* a *Completada*, y se comprobó que efectivamente el vehículo pasó a estar disponible en la tabla vehiculos. Después de la prueba, se restauraron los datos a su estado original.

Trigger actualizar_mantenimiento

El segundo trigger se activa **después de insertar** un nuevo registro en la tabla mantenimientos. Su función es actualizar automáticamente la columna ultimoMantenimiento del vehículo afectado con la fecha del nuevo mantenimiento realizado. Así se asegura que siempre se tenga un dato actualizado sobre la última revisión de cada vehículo.

Este trigger también fue probado insertando un nuevo mantenimiento preventivo que actualizó correctamente el campo ultimoMantenimiento del vehículo correspondiente.

A continuación se muestra parte del script, que **puede se consultado al completo en la carpeta de scripts del proyecto de Github**:

```
1  # Vamos a generar algunas vistas que pueden ser utiles para consultar tanto los clientes y vehiculos,
2  # como la parte de gestión de la flota y los empleados.
3  • USE carrentalx;
4
5  -- Vista para ver los clientes y los vehículos alquilados
6
7  • CREATE OR REPLACE VIEW `vista_clientes_vehiculos_alquilados` AS
8  SELECT c.idCliente, c.nombreCli, c.apellidosCli, v.matricula, v.tipo, v.marca, r.fechaInicio, r.fechaFin
9  FROM clientes c
10 JOIN reservas r ON c.idCliente = r.idCliente
11 JOIN vehiculos v ON r.idVehiculo = v.idVehiculo;
12
13 • SELECT * FROM vista_clientes_vehiculos_alquilados;
14
15 -- Una vista para ver los vehículos más reservados
16
17 • CREATE OR REPLACE VIEW `vista_vehiculos_mas_alquilados` AS
18 SELECT v.idVehiculo, v.matricula, v.tipo, v.marca, COUNT(r.idReserva) AS veces_reservado
19 FROM vehiculos v
20 JOIN reservas r ON v.idVehiculo = r.idVehiculo
21 GROUP BY v.idVehiculo
22 ORDER BY veces_reservado DESC;
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69 # Ahora vamos completar con dos trigger que complementan el que se
70 # creó anteriormente en el script de "ampliación" con la tabla salarios y salarios_anterior.
71
72
73
74 -- Trigger para actualizar la disponibilidad del vehiculo cuando se completa la reserva
75
76
77
78 DELIMITER $$
79 • CREATE TRIGGER liberar_vehiculo_al_completar
80 AFTER UPDATE ON reservas
81 FOR EACH ROW
82 BEGIN
83 -- Si el nuevo estado en reservas se actualiza a Completada se actualiza el vehiculo con campo disponible a Disponible.
84 IF NEW.estado = 'Completada' AND OLD.estado <> 'Completada' THEN
85 UPDATE vehiculos
86 SET disponible = 'Disponible'
87 WHERE idVehiculo = NEW.idVehiculo;
88 END IF;
89 END$$
90 DELIMITER ;
91
92
93
94
95
96
97
98 • -- Vamos a hacer una prueba a ver si funciona.
99 SELECT * FROM reservas;
100 • SELECT * FROM vehiculos;
101
102 -- Paso la moto Yamaha que esta No disponible a Disponible de la reserva 3, que esta Activa.
103
104 • UPDATE reservas SET estado = 'Completada' WHERE idReserva = 3 AND estado = 'Activa'; -- El trigger Funciona, cambia el campo a Disponible!!
105
106 -- Dejo los registros como estaban antes del trigger:
107
108 • UPDATE reservas SET estado = 'Activa' WHERE idReserva = 3;
109 • UPDATE vehiculos SET disponible = 'No disponible' WHERE idVehiculo = 3;
```

Pruebas y validación

Durante todo el desarrollo de la base de datos **CarRentalX** se han ido haciendo pruebas constantemente, desde el diseño hasta la implementación. No se esperó al final para revisar si todo iba bien, sino que en cada paso se hicieron pequeñas comprobaciones para asegurarse de que el sistema funcionara como se esperaba.

Desde el inicio, al crear el modelo relacional y pasarlo a MySQL, se comprobó que las claves primarias y foráneas estuvieran bien colocadas, y que los tipos de datos fueran los correctos según la información que se iba a manejar. Después, cuando se insertaron los primeros datos, se realizaron consultas básicas para ver si los registros se habían guardado bien y si las relaciones entre tablas funcionaban correctamente.

En la parte de **consultas propuestas**, se pusieron a prueba diferentes escenarios: búsquedas simples, filtros por fechas, funciones de agrupación y hasta rankings. Gracias a esto, se pudo confirmar que la base de datos respondía correctamente en situaciones reales, como ver cuáles son los vehículos más alquilados, qué clientes han hecho más reservas, o cuántos ingresos genera cada unidad.

También se crearon varias **vistas** para simplificar el acceso a datos importantes sin tener que escribir consultas largas todo el tiempo. Estas vistas fueron probadas una por una para asegurarse de que devolvían los resultados esperados, como listar los vehículos disponibles, las reservas completadas o los ingresos por vehículo. Cada vista ayudó a comprobar que la lógica de la base de datos era coherente y útil para quienes vayan a usarla.

Por otro lado, se trabajó con **triggers**, que son acciones automáticas que se ejecutan cuando pasa algo en la base de datos. Por ejemplo, se hizo un trigger que marca un vehículo como “disponible” automáticamente cuando se completa una reserva, y otro que actualiza la fecha del último mantenimiento de un vehículo cuando se registra una revisión nueva. Estos triggers se probaron con datos reales y funcionaron sin problemas, demostrando que ayudan mucho a mantener todo actualizado sin tener que hacerlo manualmente.

Con la ampliación del sistema y la gestión de salarios, también se probaron inserciones y actualizaciones junto a un trigger que guarda el historial de sueldos. Todo esto se validó con consultas para asegurarse de que no se perdía ningún dato y que todo quedaba bien registrado.

Las pruebas fueron parte del día a día del proyecto. Se hicieron a lo largo de todas las fases y sirvieron para garantizar que la base de datos es estable y capaz de funcionar correctamente en un entorno real.

Conclusiones

El desarrollo de la base de datos CarRentalX ha sido una experiencia completa y práctica, en la que se han podido aplicar y conectar distintos conceptos de diseño, modelado, implementación y optimización de bases de datos reales. Desde los primeros pasos con el modelo conceptual hasta los últimos detalles con triggers y vistas, se ha trabajado con el objetivo de construir un sistema funcional y bien organizado que puede crecer en el futuro.

A lo largo del proyecto se ha visto lo importante que es tener una buena base desde el diseño: entender bien el problema, identificar las entidades clave y definir cómo se relacionan. Gracias a esto, el modelo relacional resultante fue sólido y fácil de trasladar a MySQL. La implementación técnica también fue fluida porque se cuidó la integridad desde el principio, con claves bien definidas, tipos de datos adecuados y estructura normalizada para evitar redundancias.

Una vez creadas las tablas, se insertaron datos que permitieron poner a prueba el sistema con consultas reales. Las consultas propuestas cubrieron un montón de situaciones útiles, desde análisis básicos hasta búsquedas más complejas con filtros, ordenamientos y rankings. Esto sirvió para probar la lógica del sistema y también para afianzar el manejo del lenguaje SQL en diferentes contextos.

La inclusión de vistas ayudó a simplificar el acceso a información clave sin tener que escribir consultas largas cada vez. Algunas vistas permitieron ver datos agrupados, como ingresos por vehículo o empleados activos por sucursal, lo que aporta claridad y mejora la experiencia para quienes vayan a usar el sistema más adelante.

Además, se añadieron triggers que aportan mucha utilidad, porque automatizan tareas repetitivas o críticas, como liberar automáticamente un vehículo al completar una reserva o actualizar la fecha de último mantenimiento cuando se registra uno nuevo. Esto le da al sistema un toque inteligente, donde algunas cosas “suceden solas” sin que haya que hacerlas a mano.

También fue muy positiva la parte de **ampliación**, donde se integró la gestión de salarios con su historial.

CarRentalX ha sido una base de datos construida paso a paso, con pruebas constantes, mejoras funcionales y buenas prácticas aplicadas. Está lista para integrarse en una aplicación más grande, escalar según las necesidades del negocio, o seguir ampliándose con nuevas funcionalidades en el futuro.

Anexos

Todos los scripts utilizados a lo largo del proyecto que incluyen la creación de tablas, inserción de datos, consultas, vistas, triggers y ampliaciones, pueden ser consultados directamente en el repositorio de GitHub del proyecto. Están organizados en carpetas dentro de la ruta correspondiente:

Repositorio de GitHub:

<https://github.com/MaxiDiaz-opt/curso2024-2025-Maxi/tree/main/BASES%20DE%20DATOS%20%201%C2%BA%20DAM/PROYECTOS%20DE%20BASES%20DE%20DATOS/proyecto-maxi-diaz/>

Desde ahí se puede acceder fácilmente a todo el material que respalda cada uno de los apartados del informe.